

Prototype Report

Using T265 for VIO, and 2MP USB Camera Module for RGB observation

Introduction

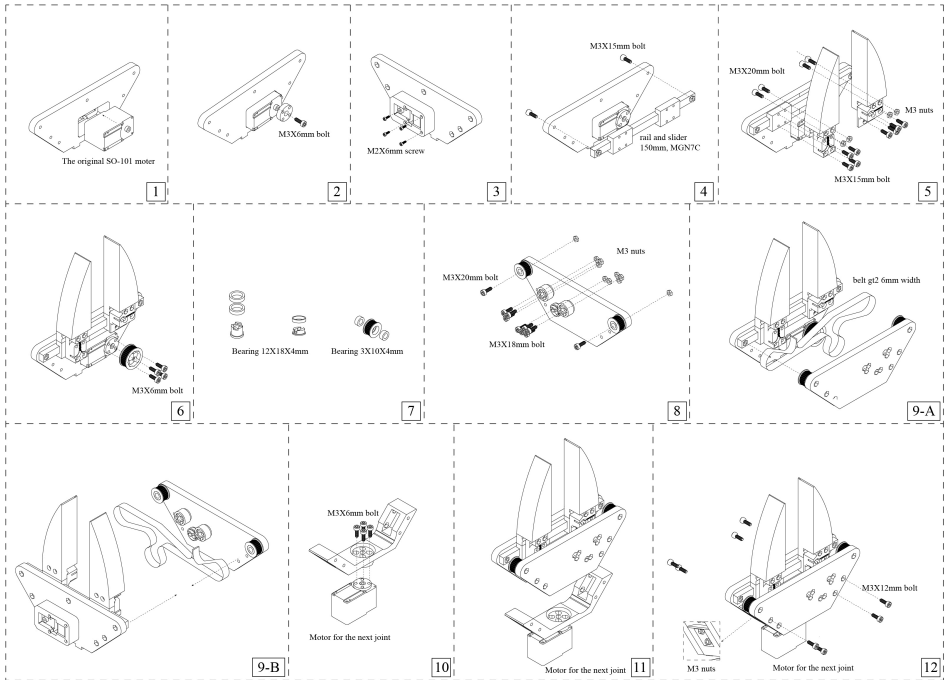
The prototype report presents the design and implementation of a low-cost uni gripper using a T265 camera for visual-inertial odometry (VIO) and a 2MP USB camera module for RGB observation. It is impressed [FastUMI](#) for simple setup. This report will discuss about the system components, the gripper design, and the implementation details.

System Components

- T265 Camera
- [2MP USB Camera Module](#)
- [Grip4SO101](#) (3d printed)
- [STS3215-12V 30KG Torque](#)
- LED
- Serial Bus Servo Driver Board
- Rasberri Pi 4

Why We use Grip4SO101

Gripper for SO101 (Installation Instruction)



Grip4SO101 directly controlled by motor, thus we can get the information of the gripper state (especially the gripper width, and power of grasping) by reading the motor gear state. Even our goal is direct plug-play style, thus it doesn't need more substantial modification to use for gripper of so101.

Implementation Details

Gripper Design

The gripper use [STS3215-12V 30KG Torque](#) to control the gripper. The gripper is 3d printed, and the gripper design is [Grip4SO101](#). We modify Grip4SO101 to add a LED indicator, a serial bus servo driver board, Rasberry Pi, hand held part like [UMI](#) and Li-Po.

How to Use the Gripper

Motor of gripper is calibrated by Homing. After that, we can control the gripper by sending the command to the motor. Simply we will make open and close button on the hand held part. Before start you need to start by clicking the start button, and finish the episode by clicking the start button.

There are three button : Start/Finish Button, Open Button, Close Button

Time Synchronization via WiFi

When collecting demonstration data from multiple gripper units simultaneously, each Raspberry Pi runs its own system clock. Even small clock differences — a few milliseconds — cause frame misalignment between grippers, which degrades the quality of imitation learning policies trained on that data. At 30 Hz capture rate, one frame period is 33.3 ms; a drift of even 10 ms means observations from two grippers no longer correspond to the same physical moment. For bimanual tasks, synchronized capture is therefore essential.

The simplest approach leverages WiFi networking already available on the Raspberry Pi 4B. Network Time Protocol (NTP) is the standard method for synchronizing clocks across networked machines. The **ntplib** PyPI package provides a pure-Python NTP client that can query any NTP server — public servers like pool.ntp.org or a local NTP server on the same WiFi network — and compute the clock offset using the classic four-timestamp exchange (t_1, t_2, t_3, t_4). At the system level, **chrony** is a modern NTP daemon that can be installed on the Raspberry Pi to continuously discipline the system clock against an NTP source. Chrony is particularly well-suited for intermittently-connected devices and achieves faster initial convergence than the older `ntpd`.

Both Raspberry Pis connect to the same WiFi network, either a standard router or a WiFi Direct peer-to-peer link. One Pi can act as a local NTP server (via `chrony` in server mode) while the other queries it as a client, eliminating dependence on internet access. The **ntplib** library is used for one-shot offset measurement before each recording session, while `chrony` handles continuous background drift correction during recording.

WiFi-based NTP synchronization typically achieves 1–5 ms accuracy on a local network, depending on WiFi congestion and Linux scheduling jitter. This is sufficient for 30 Hz capture where one frame period is 33.3 ms. For sub-millisecond requirements, dedicated RF synchronization using the **pyrf24** PyPI library with NRF24L01+ hardware achieves approximately 0.2 ms synchronization error — see the companion Time-Sync document for a detailed analysis of that approach.

VIO and Power System

T265 Visual-Inertial Odometry

The Intel RealSense T265 is a standalone visual-inertial odometry (VIO) device that provides 6DoF pose tracking. It contains two fisheye cameras (848×800 pixels each, 163-degree field of view) and a BMI055 IMU. The IMU runs at 200 Hz providing accelerometer and gyroscope data, while the fisheye cameras stream at 30 Hz. All VIO computation is performed onboard the T265's Movidius Myriad 2 VPU — the Raspberry Pi receives finished pose estimates with no CPU overhead for SLAM processing. Connection is via USB 3.0, and the cable should be kept under 30 cm for reliable data transfer.

VIO tracking confidence is reported on a scale of 0 to 3, where 3 is the highest. We require a confidence level of at least 2 for valid episodes; any episode where confidence drops below 2 is automatically discarded. The Python interface is provided by the **pyrealsense2** PyPI package, which must be built from source on ARM64 with the `FORCE_RSUSB_BACKEND` flag enabled.

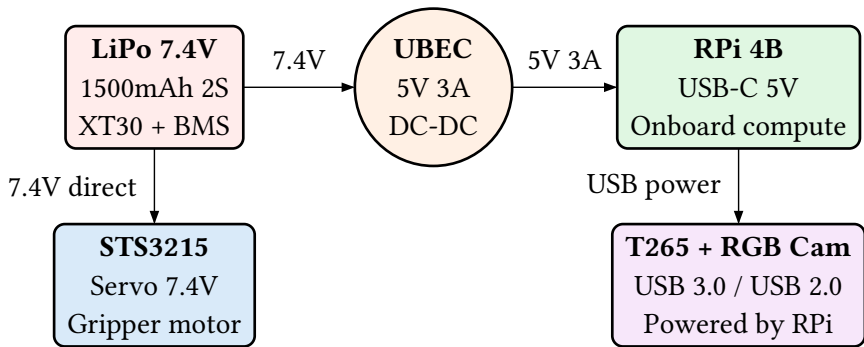


Figure 1: Power distribution. LiPo feeds servo directly at 7.4V; UBEC steps down to 5V for RPi and cameras.

Power System

The prototype is powered by a **7.4V 1500mAh 2S LiPo battery** with an XT30 connector and built-in BMS (battery management system) for safe charging and discharge protection. At 7.4V nominal, the battery provides approximately 11.1 Wh of energy. The battery directly powers the STS3215 servo, which is rated for 6–12V operation and draws peak current during gripper actuation.

The Raspberry Pi 4B requires a stable 5V supply at up to 3A. A **UBEC (Universal Battery Eliminator Circuit)** rated at 5V 3A steps down the 7.4V LiPo voltage to 5V with high efficiency (typically 90%+). The UBEC output connects to the Raspberry Pi via its USB-C power input. This is significantly more efficient and compact than a linear regulator, which would waste considerable energy as heat at the 7.4V-to-5V conversion.

Total system power consumption is approximately 10W: Raspberry Pi 4B draws 5W, T265 1.5W, STS3215 2W peak during actuation, and miscellaneous components (LED, buttons, serial driver) 1.5W. With the 1500mAh battery, expected operating time is approximately 45 minutes per charge.

Data Storage

Recording Format: HDF5

During data collection, frames are written to disk in HDF5 format using the **h5py** PyPI library. HDF5 is chosen for onboard recording because it supports efficient sequential writes, hierarchical data organization, and optional compression — all critical for sustained 30 Hz recording to a MicroSD card. Each episode is stored as a separate HDF5 file containing groups for observations, actions, and metadata.

Each frame within an HDF5 episode contains the following fields: `observation.state` as a float32 array of shape (10,) encoding position (3), quaternion (4), gripper width (1), VIO confidence (1), and side identifier (1); `observation.images.fisheye` as the T265 fisheye image; `observation.images.rgb` as the USB camera image; `action` as a float32 array of shape (10,) representing the target state for the next step; and `timestamp` as a float64 value in seconds from episode start. Extended diagnostic fields are also recorded: `nrf_rtt_ms` (float32) for the round-trip time when NRF sync is used, and `clock_offset_ms` (float32) for the estimated clock offset between peers.

Conversion to LeRobot v3.0 Format

After data collection, HDF5 episodes are transferred from the Raspberry Pi SD cards to a PC via `rsync` and converted to LeRobot v3.0 dataset format for compatibility with the LeRobot training framework. The v3.0 format organizes data into three top-level directories:

A key design choice in v3.0 is that multiple episodes are packed into single parquet and video files, reducing filesystem overhead compared to one-file-per-episode approaches. The frame-level parquet files use the following column schema:

```

dataset_name/
├── meta/
│   ├── info.json      # fps, robot_type, features, path templates
│   ├── stats.json     # global feature statistics (mean/std/min/max)
│   ├── tasks.jsonl    # task descriptions
│   └── episodes/
│       ├── chunk-000/
│       │   └── file-000.parquet # episode metadata (lengths, offsets)
├── data/
│   ├── chunk-000/
│   │   └── file-000.parquet    # multi-episode frames
└── videos/
    ├── observation.images.fisheye/
    │   ├── chunk-000/
    │   │   └── file-000.mp4    # T265 fisheye video
    ├── observation.images.rgb/
    │   ├── chunk-000/
    │   │   └── file-000.mp4    # USB RGB camera video

```

Listing 1: LeRobot v3.0 dataset directory structure. Multiple episodes are packed into single data and video files.

Column	Type	Shape	Description
observation.state	float32	(10,)	Position (3) + quaternion (4) + gripper (1) + confidence (1) + side (1)
observation.images.fisheye	video	(800, 848, 1)	T265 fisheye frame (MP4)
observation.images.rgb	video	(1080, 1920, 3)	USB RGB camera frame (MP4)
action	float32	(10,)	Target state for next step
timestamp	float64		Seconds from episode start
nrf_rtt_ms	float32		Round-trip time measurement (ms)
clock_offset_ms	float32		Estimated clock offset (ms)
episode_index	int64		Episode identifier
frame_index	int64		Frame position within episode
index	int64		Global frame index across dataset
task_index	int64		Links to tasks.jsonl

Table 1: Parquet columns per frame. Extended with sync diagnostics for post-hoc timing analysis.

The conversion pipeline uses **h5py** to read the recorded HDF5 files, **pyarrow** to write parquet files, and **pandas** for tabular data manipulation during the merge step. Video frames (fisheye and RGB) are encoded to MP4 using ffmpeg. The **lerobot** PyPI package provides utilities for dataset validation, statistics computation (mean, std, min, max across all features), and direct integration with policy training scripts such as Diffusion Policy and ACT.

TODO

Hardware Assembly

Goal: Assemble and wire one complete prototype gripper unit.

- Print Grip4SO101 gripper housing and hand-held ergonomic handle in PLA. Print TPU fingertips for compliant grasping.
- Mount STS3215 servo into Grip4SO101 housing. Verify smooth jaw open/close motion across full range.
- Mount T265 camera onto gripper housing with forward-facing orientation. Use TPU vibration dampening pads between camera and mount.
- Mount 2MP USB camera module on gripper housing with forward-facing orientation for RGB observation.
- Wire LiPo 7.4V 1500mAh battery with XT30 connector. Connect 7.4V directly to STS3215 servo power input.
- Connect UBEC (5V 3A) input to LiPo 7.4V output. Connect UBEC 5V output to Raspberry Pi USB-C power input.
- Install LED indicator and connect to RPi GPIO27 with 330 Ω series resistor.
- Install three buttons (Start/Finish, Open, Close) and connect to RPi GPIO with pull-up resistors.
- Verify full assembly powers on: RPi boots, T265 LED active, STS3215 responds to commands, LED toggles via GPIO.

Software Environment

Goal: Prepare the Raspberry Pi software environment with all required libraries.

- Flash Ubuntu 22.04 Server 64-bit onto MicroSD 64GB A2 card for RPi 4B.
- Build librealsense v2.53.1 from source with FORCE_RSUSB_BACKEND and Python bindings enabled on RPi.
- Add T265 udev rule to disable USB autosuspend for vendor 8087, product 0b37.
- Enable GPIO UART on /dev/ttyAMA0 for STS3215 serial communication.
- Install PyPI packages: pyrealsense2 (built from source), pyserial for STS3215 UART communication.
- Install PyPI packages for time sync: ntplib for NTP offset measurement. Configure chrony daemon for continuous clock discipline.
- Install PyPI packages for data recording: h5py for HDF5 storage, pyarrow and pandas for parquet conversion.
- Install PyPI packages for training pipeline: lerobot for dataset utilities, opencv-python for image processing.
- Verify pyrealsense2 imports and T265 outputs 6DoF pose. Verify STS3215 returns encoder values via pyserial.
- Configure WiFi connection between both RPis (either shared router or WiFi Direct peer-to-peer).

Single Gripper Recording

Goal: Record one gripper's data (pose + fisheye + RGB + gripper width) to SD card at 30 Hz.

- Implement T265 callback thread: buffer pose at 200 Hz, capture fisheye at 30 Hz via pyrealsense2.
- Implement USB RGB camera capture thread at 30 Hz using opencv-python VideoCapture.
- Implement STS3215 UART polling thread at 100 Hz via pyserial. Read encoder position, velocity, and load.
- Build frame synchronizer: on each 30 Hz tick, latch nearest T265 pose (max error ± 2.5 ms at 200 Hz), nearest fisheye frame, nearest RGB frame, and nearest STS3215 reading (max error ± 5 ms at 100 Hz).
- Implement episode recorder: GPIO button press starts/stops recording. LED indicates recording state (solid on = recording, blinking = idle).
- Write frame bundles to HDF5 file on SD card using h5py. Include `observation.state(10,)`, `observation.images.fisheye`, `observation.images.rgb`, `action(10,)`, and `timestamp`.
- Validate sustained 30 Hz write with no dropped frames over a 60-second recording session.
- Build playback visualizer: display fisheye + RGB side by side with pose trajectory overlay and gripper width indicator.

Time Synchronization

Goal: Achieve sub-5 ms clock synchronization between two Raspberry Pis.

- Install and configure chrony on both RPis. Designate one Pi as local NTP server, the other as client.
- Implement pre-recording offset measurement using ntplib: query local NTP server 10 times, apply median-based outlier rejection, compute final offset estimate.
- Measure and validate WiFi NTP sync accuracy: expected 1–5 ms. Log RTT and offset for each measurement round.
- Implement alternating UDP trigger protocol for frame-level synchronization between two grippers (even steps from Peer A, odd steps from Peer B).
- Implement 28-byte UDP sync packet: magic, msg_type, step, episode, flags, sender_id, timestamp, sync_t2, checksum.
- Add packet loss detection via step number gap analysis. Insert empty frames (copy previous state) for lost steps.
- Stress test: 10-minute continuous bimanual recording. Verify A/B step counts match exactly and average RTT stays below 5 ms.
- Optional upgrade: implement NRF24L01+ synchronization using pyrf24 for sub-millisecond accuracy (see Time-Sync companion document).

Data Conversion and LeRobot Integration

Goal: Convert recorded HDF5 episodes to LeRobot v3.0 format and validate for training.

- Implement HDF5-to-parquet conversion script: read h5py episodes, write data/chunk-000/file-000.parquet with all required columns using pyarrow.
- Encode fisheye and RGB video streams to MP4 files under videos/observation.images.fisheye/ and videos/observation.images.rgb/ directories.
- Generate meta/info.json with fps (30), robot_type, feature definitions, and path templates for the dataset.
- Compute and write meta/stats.json with global statistics (mean, std, min, max) for all numeric features using pandas.
- Generate meta/tasks.jsonl with task descriptions for each demonstration type (e.g., pick-and-place, handover).
- Write meta/episodes/chunk-000/file-000.parquet with episode metadata (lengths, start indices, task assignments).
- Implement bimanual merge script: match episodes from Peer A and Peer B by absolute step number, zip into unified dataset.
- Validate converted dataset loads correctly with lerobot Python package. Verify all columns, video playback, and statistics match expectations.

Pilot Data Collection and Evaluation

Goal: Collect demonstration episodes and validate end-to-end pipeline.

- Collect 10 single-gripper pick-and-place episodes (cups, blocks) to validate recording pipeline.
- Review recorded data quality: check VIO confidence stays at 2 or above, verify no dropped frames, confirm gripper width readings are consistent.
- Collect 20+ bimanual demonstration episodes with two synchronized grippers.
- Transfer data from both RPi SD cards to PC via rsync. Run bimanual merge script.
- Convert merged dataset to LeRobot v3.0 format. Run lerobot dataset validation.
- Train a Diffusion Policy or ACT policy on the collected dataset as proof-of-concept.
- Evaluate trained policy qualitatively: does it reproduce the demonstrated trajectories and gripper actions?